Ben Antonellis, Alex Nguyen, Gabe Smith

SmartWaterMeter Test Report

Preface

Detailed below are System Testing and Metamorphic Testing for the SmartWaterMeter project. The bulk of these tests are on our utility functions, as we only have one source of input for the actual program. We cannot collect more usage data, as we have only been given permission for a very specific period of time.

Test Case #1

- **Function:** OutputPlaintext
- **Requirements:** The function must take in a vector of vectors of integers, then output them correctly in the specified format.
- **Input/Output:** resultDec1 and resultDec2 from Step2_TA1.cpp. The expected output was consistent with the decryption and encoding performed in this step.

Test Case #2

- **Function:** OutVector
- **Requirements:** This function must take in a vector of integers, then output them in a colorful format from the beginning to end.
- **Input/Output:** vecLog and vecRecLog from Step1_CS1.cpp. The expected output was consistent with the storage from iterating (i, rowSize). The output is dependent on the table size specified in SGSimulation.hpp.

Test Case #3

- **Function:** PrintParameters
- **Requirements:** The function takes in a shared pointer reference of a SEAL context, then outputs diagnostic information about the current context. This is useful for ensuring we did our setup correctly for each Step#_###.cpp file.
- **Input/Output:** The output was consistent with each parameter set for each Step#_###.cpp file that we are using. The encryption parameters were what we specified and the modulus information was correct given the Brakerski-Fan-Vercauteren calculation scheme.

Test Case #4

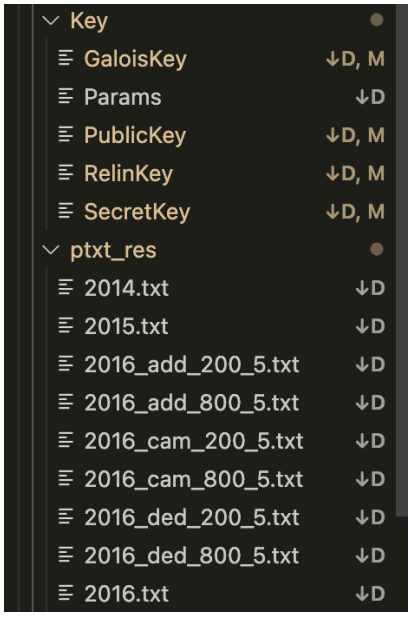
- **Function:** ShiftWork (index)
- **Requirements:** This function takes in a vector of integers as query, an index as an integer, and the current number of slots as an integer, and shifts the indices by the specified index, returning a new vector with the shifted values
- **Input/Output:** queryAM1 and queryHM1 from Step2_TA1.cpp. This function was only testable using the index function, not the random function, because we cannot test randomness. However, given the index numbers we were using for the number of meters defined in SGSimulation.hpp, the output delivered a correct encryption scheme.

Test Case #5

- **Function:** LoadKey
- **Requirements:** The function must take in a SEAL context and a filepath as a string pointing toward the current key directory.
- **Input/Output:** The input was created using the CreateContextFromParams (tested below), then passed in with a file path defined in SGSimulation.hpp. We know these tests have passed because the encryptor must be given a valid public key, and we create our keys in KeyGen.cpp (tested below).

Test Case #6

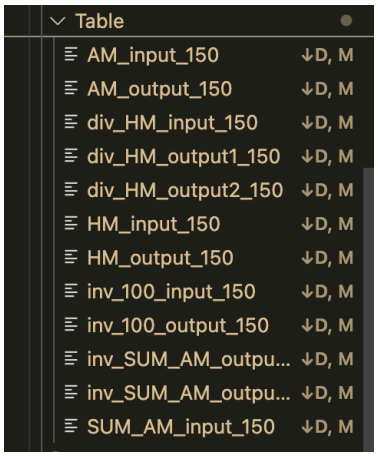
- **File:** KeyGen.cpp
- **Requirements:** The function must generate a Public Key, Secret Key, Galois Keys, and Relin Keys. They must be stored in the Keys directory, and must be loadable from different contexts as long as they use the same encryption parameters. This file aims to reduce the complexity of re-generating keys for every step.
- **Input/Output:** We are limited as to how liberally we can test this function; setting incorrect encryption parameters for different moduluses will result in incorrect encryption. Because of the scheme we are using, we must use: 8192 poly modulus degree, 8192 coefficient degree (the BFV default degree), and 786433 plain modulus degree. The testable context of this file is used alongside PrintParameters (tested above).




```
================================
subprocess.getstatusoutput(./bin/KeyGen)
================================
Batching Enabled: true
Plaintext matrix row size: 4096
Slot nums = 8192
Save public key and secret key...
Saving completed.
================================
```

```
subprocess.getstatusoutput(bin/MakeEncTab_1)
============================
Precision = 32
////////////////////////////
InputArith, from 18966 to 41759.3.
InputHarm, from 606911 to 1.3363e+06.
InputArith table size is: 22795
InputHarm table size is: 21218

HM input from 38873 to 17656
HM output from 2023 to 4454
-----
AM input from 18965 to 41759
AM output from 2023 to 4454
AM out size: 22795, HM out size: 21218
-----
input 1/sam from 48552 to 106896
output 1/sam from 22115 to 10045
Size: 58345
-----
input 1/shm from 48552 to 106896
Size: 58345
Divided HM output table.
```



Metamorphic Testing

Computational Accuracy - The AM, HM, and HM/AM ratio calculated on plaintext data should be consistent with the decrypted values obtained after performing the same operations on encrypted data

Test Passed - Fully Homomorphic Encryption basic arithmetic and functional operations produced same results as plaintext data on ciphertext data

Scaling Relation - Scaling the input data shout effect the Arithmetic Mean, as well as the Harmonic Mean should also be scaled similarly

Test Passed - Scaling the input effected the arithmetic mean, which also influenced the Harmonic mean similarly, however the HM/AM ratio remained the same